Text-to-SQL Empowered by Large Language Models: A Benchmark Evaluation

Dawei Gao*
Alibaba Group
gaodawei.gdw@alibaba-inc.com

Haibin Wang*
Alibaba Group
binke.whb@alibaba-inc.com

Yaliang Li
Alibaba Group
yaliang.li@alibaba-inc.com

Xiuyu Sun
Alibaba Group
xiuyu.sxy@alibaba-inc.com

Yichen Qian
Alibaba Group
yichen.qyc@alibaba-inc.com

Bolin Ding
Alibaba Group
bolin.ding@alibaba-inc.com

Jingren Zhou
Alibaba Group
jingren.zhou@alibaba-inc.com

arXiv:2308.15363v3 [cs.DB] 20 Sep 2023

ABSTRACT

Large language models (LLMs) have emerged as a new paradigm for Text-to-SQL task. However, the absence of a systematical benchmark inhibits the development of designing effective, efficient and economic LLM-based Text-to-SQL solutions. To address this challenge, in this paper, we first conduct a systematical and extensive comparison over existing prompt engineering methods, including question representation, example selection and example organization, and with these experimental results, we elaborate their pros and cons. Based on these findings, we propose a new integrated solution, named DAIL-SQL, which refreshes the Spider leaderboard with 86.6% execution accuracy and sets a new bar.

To explore the potential of open-source LLM, we investigate them in various scenarios, and further enhance their performance with supervised fine-tuning. Our explorations highlight open-source LLMs' potential in Text-to-SQL, as well as the advantages and disadvantages of the supervised fine-tuning. Additionally, towards an efficient and economic LLM-based Text-to-SQL solution, we emphasize the token efficiency in prompt engineering and compare the prior studies under this metric. We hope that our work provides a deeper understanding of Text-to-SQL with LLMs, and inspires further investigations and broad applications.

1 INTRODUCTION

Text-to-SQL, as one challenging task in both natural language processing and database communities, maps natural language questions on the given relational database into SQL queries [7, 15]. Most previous works [14, 19, 20, 42, 50] focus on extracting the question-to-SQL patterns and generalizing them by training an encoder-decoder model with Text-to-SQL corpus. In recent years, large language models (LLMs) have emerged as a new paradigm for Text-to-SQL [22, 34, 41]. Notably, equipped with GPT-4 [26], Pourreza et al. [31] achieved the first place in Spider leaderboard [3] with 85.3% execution accuracy. Different from prior studies, the core problem in LLM-based Text-to-SQL solution is how to prompt LLM to generate correct SQL queries, namely prompt engineering. Such prompt engineering involves question representations [5, 10, 28, 31], examples selection [11, 24, 25], and example organization [11].

Text-to-SQL prompt engineering needs a systematic study. Although prior studies have made remarkable progress, there still lacks a systematic study for prompt engineering in LLM-based Text-to-SQL solutions. Specifically, for question representation, most existing research textualize structured knowledge as schema, and further add task instructions and foreign keys to form prompts [16, 25]. Besides, some studies [5, 25] represent tables as several "*CREATE TABLE*" SQL statements, and prompt LLMs to answer the target question in comments. However, even with similar representation, their detailed task instructions can lead to significant performance gap. For example, in OpenAI's official Text-to-SQL demo [28], they employ the pound sign "#" to differentiate prompt from response, yielding an impressive performance [22]; If such a sign is removed, the performance will significantly drop. Therefore, there are burgeoning demands for a systematic study over different representations and examine how to work well with LLMs. Regarding example selection, a common practice is to encode the most similar examples in the same representation with the target question [5, 22, 25]. Nan et al. [25] further underline the importance of diversity in example selection. While for organization, most prior studies represent examples with full information, including instruction, schema, question and ground truth SQL queries. Besides, Guo et al. [11] only keep SQL queries in the selected examples to guide the LLM with less tokens. Together with different LLMs' preferences, the optimal selection and organization strategies in LLM-based Text-to-SQL solution remain ambiguous. Therefore, a systematical study on prompt engineering, spanning different LLMs, question representations, example selection and organizations, is highly anticipated.

The potential of open-source LLMs is underexplored. Very recently, open-source LLMs are constantly expanding and show remarkable advancement in programming, mathematical reasoning, and text generation tasks. However, previous Text-to-SQL research primarily focuses on OpenAI LLMs, leaving open-source LLMs unstudied. Besides, compared with OpenAI LLMs, open-source ones generally have limited functionality in understanding context and generating coherent response. Thus, a critical challenge for open-source LLMs is to further enhance their performance in Text-to-SQL, which can be achieved by supervised fine-tuning.

Prompt efficiency remains a challenging open question. In LLM-based Text-to-SQL, another critical challenge is efficiency. The reason is that most prior studies focus on OpenAI LLMs, and calling their APIs are expensive, time-consuming and restricted in rate limits [27], especially for in-context learning prompts with multiple examples. However, the prior studies may not well tackle

*Co-first authors.

this challenge. Specifically, based on the observed inverted-U shape in execution accuracy with respect to prompt length, Chang et al. [5] conjectures that LLMs may have a sweet spot in terms of prompt length, but leaves exploring efficient prompt engineering a challenging open question.

In light of above challenges, we focus on providing a comprehensive, systematical and fair benchmark for LLM-based Text-to-SQL. Specifically, our benchmark discusses both the effectiveness and efficiency of various prompt engineering strategies, as well as the feasibility of open-source LLMs. They are detailed as follows.

To provide a systematical and in-depth understanding of Text-to-SQL prompt engineering, we empirically evaluate several strategies from prior studies. First, we compare several typical question representations in zero-shot scenario with different LLMs, and identify their pros and cons. After that, we investigate example selection and organization strategies in few-shot scenario. For example selection, we compare different selection strategies and further verify the hypothesis that LLMs learn from the mappings between question and SQL skeleton. Regarding example organization, we explore the option of displaying full information, solely SQL queries or question-SQL pair.

After that, we highlight the potential of open-source LLMs in both in-context learning and supervised fine-tuning. Specifically, we empirically study various open-source LLMs with different prompt engineering strategies, and observe the significant benefits of increasing scale of LLMs and having a good alignment [29]. To further enhance their performance, we fine-tune and evaluate open-source LLMs using various representations. With this comparison, we demonstrate that similar to in-context learning, representation strategy is also critical for supervised fine-tuning. These explorations underline the potential of an effective solution for Text-to-SQL. Moreover, after fine-tuning we also observe a decrease in in-context learning capability, which requires further study. We believe these explorations will benefit practical Text-to-SQL applications.

Towards a more economic and efficient solution, we further evaluate different strategies in terms of token efficiency. Such evaluation aims at searching for a cost-effective strategy, which is supposed to achieve considerable performance with less tokens. To fulfill such goal, we consider token efficiency in the whole process of prompt engineering, including choices for question representation, example selection and organization.

Last but not least, our integrated solution, named DAIL-SQL, refreshes the Spider leaderboard with 86.6% execution accuracy, and wins the first place. Compared with previous solutions, DAIL-SQL encodes structure knowledge as SQL statements, selects examples based on their skeleton similarities and removes cross-domain knowledge from examples for token efficiency. Before DAIL-SQL, the state-of-the-art performance in the Spider leaderboard is 85.3% [31]. Therefore, our solution sets a new bar, and hope our comprehensive study will inspire more further works.

Contribution Our main contributions and results are summarized as follows:

- We conduct a comprehensive comparison for prompt engineering methods, including five question representations, four example selection strategies and three example organization strategies. With such a comparison, we methodically dissect each strategy to identity its merits and demerits. We further propose a new integrated solution, named DAIL-SQL, and it refreshes the Spider leaderboard with 86.6% execution accuracy. Notably, this performance surpasses the best state-of-the-art solution by 1.3%.
- To the best of our knowledge, we are the first to explore open-source LLMs for both in-context learning and supervised fine-tuning for Text-to-SQL task. Specifically, we point out the importance of representation in supervised fine-tuning, and the decrease of in-context learning capability after fine-tuning, both of which need further studies.
- We also empirically compare different representation and example organization strategies in terms of token efficiency, which provides practical guidance for real-world Text-to-SQL applications.

2 PRELIMINARY

Text-to-SQL aims at automatically translating natural language questions into SQL queries. It bridges the gap between non-expert users and database systems, greatly improves the efficiency of data processing, and contributes to a wider range of applications such as intelligent database service, automatic data analysis and database question-answering. However, Text-to-SQL is still a quiet challenging task, due to the difficulty in fully understanding natural language questions and generating correct SQL queries [15, 33].

Extensive studies of Text-to-SQL have been conducted in both database and natural language processing communities. Early studies treat Text-to-SQL as a sequence-to-sequence task, and focus on training machine learning models with an encoder-decoder architecture [4, 30, 32]. With rapid advancement of deep learning, numerous techniques are applied to help Text-to-SQL task, such as attention mechanism [23], graph representation [14, 20, 32, 42, 46, 50], syntax parsing [12, 19, 35, 43], etc. Besides, to narrow the gap between Text-to-SQL research and its real-world deployment, numerous large-scale benchmark datasets have been released, including WikiSQL [52], Spider [47], KaggleDBQA [18], BIRD [21] etc. With these great efforts, the research communities have made impressive progress in Text-to-SQL.

Recently, large language models (LLMs), such as GPT-4 [26] from OpenAI and LLaMA [39] from Meta, have emerged as a milestone for natural language processing and machine learning. Different from general machine learning model, LLMs are pre-trained on massive text corpus, which can perform various natural language tasks. The basic operating principle is to gradually produce the next word that has the highest probability based on the input prompt [48]. Therefore, to tackle Text-to-SQL task with LLMs, the core is to find the optimal prompt, also known as prompt engineering [22, 25].

Specifically, according to number of examples provided in prompt, prompt engineering are classified into two scenarios: zero-shot scenario and few-shot scenario. In zero-shot scenario, no example is provided, and the main challenge is to represent the natural language question effectively, including incorporating relevant information such as the corresponding database schema [5, 10, 22, 41]. In this paper, we refer it as **question representation**. While in

Question Representation	INS	RI	FK	Ref.	LLMs	EX (%)
BS $_P$	✗	✗	✗	[31]	-	-
TR $_P$	✓	✗	✗	[25]	CODE-DAVINCI-002	69.0
OD $_P$	✓	✓	✗	[22]	GPT-3.5-TURBO	70.1
				[31]	GPT-4	64.9
CR $_P$	✓	✗	✓	[25]	CODE-DAVINCI-002	75.6
				[5]	CODE-DAVINCI-002	71.8
				[5]	GPT-3.5-TURBO	70.7
AS $_P$	✓	✗	✗	[38]	-	-

Table 1: Question representations in existing works, as well as their reported execution accuracy (EX) in zero-shot scenario. The Instruction (INS), Rule Implication (RI) and Foreign Key (FK) are possible components in a prompt. INS is the task description, such as "Write a SQL to answer the question". RI is the guiding statement, such as "Complete sqlite SQL query only and with no explanation". FK is the foreign key information of the database.

few-shot scenario, a limited number of examples are available, thus besides question representation, we also need to study how to select the most helpful examples and organize them in the prompt appropriately. In natural language processing, the above progress that LLMs learn from contextual examples is referred as **in-context learning** [9]. In this paper, we will discuss in-context learning in the scope of example selection and example organization. Although LLMs are demonstrated to be effective in both zero-shot and few-shot scenarios in prior studies [5, 16, 22, 25, 37], their performances can be further enhanced by **supervised fine-tuning**, which further tune LLMs using additional Text-to-SQL instances to make it more suitable for specific downstream task. However, it is worth noting that despite the extensive research on prompt engineering for Text-to-SQL, there is a scarcity of studies exploring the supervised fine-tuning of LLMs for Text-to-SQL [37], leaving this area as an open question.

In summary, question representation, in-context learning, together with supervised fine-tuning are three essential knobs in large language model based Text-to-SQL. In this paper, we will provide a systematical study and discussion about them.

3 METHODOLOGY

As stated above, in this paper we focus on question representation, in-context learning and supervised fine-tuning. In this section, we provide formal definitions for these three problems, survey their existing solutions systematically, and point out the potential issues in existing techniques. To address these issues, we propose a new Text-to-SQL prompt engineering method, named DAIL-SQL, which refreshes the best performance in Spider leaderboard with 86.6% execution accuracy.

3.1 Question Representation

In this section, we first discuss question representations under zero-shot scenario for Text-to-SQL. Considering a target question q in natural language on certain database $\mathcal{D}$, the target of question representation is to maximize the possibility of LLM $\mathcal{M}$ generating the correct SQL s^* as follows:

$$\max_{\sigma} \quad \mathbb{P}_{\mathcal{M}}(s^*|\sigma(q, \mathcal{D})),$$

where function $\sigma(\cdot,\cdot)$ decides representation for target question q, with the useful information from the schema of database $\mathcal{D}$. Besides, $\sigma(\cdot,\cdot)$ also can include information such as instruction statement, rule implication and foreign key.

Follow the above definition, we survey different choices of σ in zero-shot scenario and choose four most representative ones from literature. In addition, we also include the question representation used in Alpaca [38] since it's popular in supervised fine-tuning. Table 1 summarizes these five representation methods and lists their reported details from their original papers.

```
Table continents, columns = [ContId, Continent]
Table countries, columns = [CountryId, CountryName,
↳ Continent]
Q: How many continents are there?
A: SELECT
```

Listing 1: Example of Basic Prompt

```
Given the following database schema:
continents: ContId, Continent
countries: CountryId, CountryName, Continent

Answer the following: How many continents are there?
SELECT
```

Listing 2: Example of Text Representation Prompt

- **Basic Prompt** (BS $_P$). Basic Prompt [31] is a simple representation shown in Listing 1. It is consisted of table schemas, natural language question prefixed by "*Q:* " and a response prefix "*A: SELECT*" to prompt LLM to generate SQL. In this paper we named it as Basic Prompt due to its absence of instructions.
- **Text Representation Prompt** (TR $_P$). As shown in Listing 2, Text Representation Prompt [25] represents both schema and question in natural language. Compared with Basic Prompt, it adds instruction at the very beginning of prompt to guide LLMs. In [25], it achieves 69.0% execution accuracy on Spider-dev in zero-shot scenario.
- **OpenAI Demostration Prompt** (OD $_P$). The OpenAI Demostration Prompt (Listing 3) is first used in OpenAI's official Text-to-SQL demo [28], and evaluated in [22, 31]. It's consisted of instruction, table schemas, and question, where all information are commented by pound sign "#". Compared with Text Representation Prompt, the instruction in OpenAI Demostration Prompt is more specific with a rule, "*Complete sqlite SQL query only and with no explanation*", which we will further discuss in the Sec. 4.3 along with experimental results.
- **Code Representation Prompt** (CR $_P$). The Code Representation Prompt [5, 25] presents Text-to-SQL task in SQL syntax. Specifically, as shown in Listing 4, it directly presents "*CREAT TABLE*" SQLs, and prompts LLM with natural language question in comments. Compared with other representations, CR $_P$ stands out due to its ability to

```
### Complete sqlite SQL query only and with no explanation
### SQLite SQL tables, with their properties:
#
# continents(ContId, Continent)
# countries(CountryId, CountryName, Continent)
#
### How many continents are there?
SELECT
```

Listing 3: Example of OpenAI Demostration Prompt

```
/* Given the following database schema: */
CREATE TABLE continents(
    ContId int primary key,
    Continent text,
    foreign key(ContId) references countries(Continent)
);

CREATE TABLE countries(
    CountryId int primary key,
    CountryName text,
    Continent int,
    foreign key(Continent) references continents(ContId)
);

/* Answer the following: How many continents are there? */
SELECT
```

Listing 4: Example of Code Representation Prompt

```
Below is an instruction that describes a task, paired with an input that provides further context. Write a response that appropriately completes the request.

### Instruction:
Write a sql to answer the question "How many continents are there?"

### Input:
continents(ContId, Continent)
countries(CountryId, CountryName, Continent)

### Response:
SELECT
```

Listing 5: Example of Alpaca SFT Prompt

provide comprehensive information necessary for database creation, such as column types and primary/foreign keys. With such a representation, [25] correctly predicts about 75.6% SQLs with LLM CODE-DAVINCI-002.

- **Alpaca SFT Prompt** (AS $_P$). The Alpaca SFT Prompt is a prompt designed for supervised fine-tuning [38]. As shown in Listing 5, it prompts LLM to follow instruction and finish task according to the input context in Markdown format. We include it to examine its effectiveness and efficiency in both prompt engineering and supervised fine-tuning scenarios.

As shown in Table 1, different representations are experimented with different LLMs, and integrated in different frameworks, making it difficult to compare them fairly and effectively. Additionally, the specific roles played by individual components such as foreign key information and rule implication remain unclear. Consequently, it is essential to conduct a systematical study to better understand question representations, and investigate their advantages and disadvantages through a fair comparison.

3.2 In-Context Learning for Text-to-SQL

The above question representation methods enable LLMs to directly output desired SQLs by zero-shot learning. However, LLMs can perform better for Text-to-SQL through in-context learning, in which only a few examples are provided in the input prompts. Therefore, in this subsection, we discuss the keys of in-context learning, that are example selection and example organization. We first give a formulation of in-context learning to ease the further discussions.

In Text-to-SQL, given a set of triples $Q = \{(q_i, s_i, \mathcal{D}_i)\}$, where q_i and s_i are natural language question and its corresponding SQL query on database $\mathcal{D}_i$, the target of in-context learning for Text-to-SQL is to maximize the possibility of LLM $\mathcal{M}$ generating the correct SQL s^* on the target question q and database $\mathcal{D}$ as follows:

$$\begin{aligned} \max_{Q',\sigma} \quad & \mathbb{P}_{\mathcal{M}}(s^* | \sigma(q, \mathcal{D}, Q')), \\ \text{s.t.} \quad & |Q'| = k \quad \text{and} \quad Q' \subset Q, \end{aligned}$$

where function $\sigma(\cdot,\cdot,\cdot)$ decides representation for target question q, with the useful information from the schema in database $\mathcal{D}$ and k examples selected from Q. In this paper, we focus on *cross-domain Text-to-SQL*, which means the target database $\mathcal{D}$ is not included among the databases $\mathcal{D}_i$ mentioned in Q., i.e., $\mathcal{D} \notin \{\mathcal{D}_i | (q_i, s_i, \mathcal{D}_i) \in Q\}$.

In-context learning for Text-to-SQL involves selecting the most helpful examples Q' and deciding how to organize the information of these selected examples into prompt. Next we discuss these two sub-tasks: example selection and example organization.

3.2.1 Example Selection. We summarize various example selection strategies in prior studies as follows.

- **Random**. This strategy randomly samples k examples from the available candidates. Previous works [11, 24, 25] have adopted it as a baseline for example selection.
- **Question Similarity Selection** (QTS $_S$). QTS $_S$ [24] chooses k examples with the most similar questions. Specifically, it embeds both example questions in Q and the target question q with a pre-trained language model. Then it applies a pre-defined distance measure, such as the Euclidean distance or negative cosine similarity, to each example-target pair. Finally kNN algorithm is leveraged to select k examples from Q that closely match the target question q.
- **Masked Question Similarity Selection** (MQS $_S$). For cross-domain Text-to-SQL, MQS $_S$ [11] eliminates the negative influence of domain-specific information by replacing table names, column names, and values in all questions with a mask token, and then compute the similarities of their embedding with kNN algorithm.
- **Query Similarity Selection** (QRS $_S$). Instead of using the target question q, QRS $_S$ [25] aims to select k examples that are similar to target SQL query s^*. Specifically, it employs a preliminary model to generate SQL query s' using target

```
/* Given the following database schema: */
${DATABASE_SCHEMA}
/* Answer the following: How many authors are there? */
SELECT count(*) FROM authors

/* Given the following database schema: */
${DATABASE_SCHEMA}
/* Answer the following: How many farms are there? */
SELECT count(*) FROM farm

${TARGET_QUESTION}
```

Listing 6: Example of Full-Information Organization.

```
/* Some SQL examples are provided based on similar
↳ problems: */
SELECT count(*) FROM authors

SELECT count(*) FROM farm

${TARGET_QUESTION}
```

Listing 7: Example of SQL-Only Organization.

```
/* Some example questions and corresponding SQL queries
↳ are provided based on similar problems: */
/* Answer the following: How many authors are there? */
SELECT count(*) FROM authors

/* Answer the following: How many farms are there?. */
SELECT count(*) FROM farm

${TARGET_QUESTION}
```

Listing 8: Example of DAIL Organization.

question q and database D, where this generated s' can be regarded as an approximation of s^*. Then it encodes queries from examples into binary discrete syntax vectors according to their keywords. After that, it chooses k examples by considering both similarity to the approximated query s' and diversity among selected examples.

Above strategies focus on selecting examples using only target question or query. However, according to prior studies [9], in-context learning is essentially learning from analogy. In the case of Text-to-SQL, the objective is to generate queries that match the given questions, thus LLMs are supposed to learn the mapping from questions to SQL queries. Therefore, we point out that during example selection, taking both question and SQL queries into consideration may benefit Text-to-SQL task. We will further discuss it in Sec. 3.3.

3.2.2 Example Organization. The example organization plays a pivotal role in determining what information of the above selected examples will be organized into the prompt. We summarize existing strategies in prior studies into two categories, Full-Information Organization and SQL-Only Organization, as demonstrated in Listing 6 and Listing 7. In these examples, *${DATABASE_SCHEMA}* represents the database schema, and *${TARGET_QUESTION}* stands for the question representation in Listing 4.

- **Full-Information Organization** (FI_O). FI_O [5, 25] organizes examples in the same representation with the target question. As shown in Listing 6, examples are structured identically to the target question, and the only difference is that instead of the "*SELECT*" token at the end, the selected examples have the corresponding SQL queries after "*SELECT*".
- **SQL-Only Organization** (SO_O). SO_O [11] includes only SQL queries of the selected examples with a prefix instruction in the prompt, as demonstrated in Listing 7. Such organization aims at maximizing the number of examples with limited token length. However, it removes the mapping information between questions and corresponding SQL queries, and such information can be useful, which we will demonstrate later.

In summary, FI_O includes the full information of examples, which ensures the quality; while SO_O only keeps SQL queries to accommodate more examples, which prefers the quantity. We wonder if there exists a better trade-off between quality and quantity in example organization , which can further benefit the Text-to-SQL task.

3.3 DAIL-SQL

To address the aforementioned issues in example selection and organization, in this subsection, we present a novel Text-to-SQL method named DAIL-SQL.

For example selection, inspired by MQS_S and QRS_S, we proposed **DAIL Selection** (DAIL_S), considering both questions and queries to select candidates. Specifically, DAIL Selection first masks domain-specific words in both target question q and example questions q_i in the candidate set $\boldsymbol{Q}$. It then ranks the candidate examples based on the Euclidean distance between the embeddings of masked q and q_i. Simultaneously, it calculates the query similarity between the pre-predicted SQL query s' and s_i in $\boldsymbol{Q}$. Finally, the selection criterion prioritizes the sorted candidates by question similarity with a query similarity greater than a predefined threshold τ. In this way, the selected top k examples have good similarity with both question and query.

To preserve the mapping information between questions and SQL queries and also improve the token efficiency, we propose a new example organization strategy **DAIL Organization** (DAIL_O) to trade-off in terms of quality and quantity. Specifically, DAIL_O presents both questions q_i and corresponding SQL queries s_i, as illustrated in Listing 8. As a compromise between FI_O and SO_O, DAIL_O reserves the question-SQL mapping, and reduces the token length of examples by removing token-cost database schema.

In DAIL-SQL, we adopt CR_P as our question representation. The reason is that compared with other representations, CR_P contains full information of the database, including primary and foreign keys, which may offers more useful information for LLMs, such as foreign keys for the prediction of "JOIN" clauses. Besides, pre-trained on extensive coding corpora, LLMs could better understand the prompt in CR_P without too much additional effort.

In summary, DAIL-SQL utilizes CR_P as the question representation, selects examples based on information from both question and query, and organizes them to keep question-to-SQL mappings.

In such prompt design, LLMs could work better for Text-to-SQL task, and in Spider leaderboard, the proposed DAIL-SQL refresh the performance with 86.2% execution accuracy.

Note DAIL-SQL is a flexible LLM-based Text-to-SQL solution, which can be further extended and integrated with other components easily. For example, to improve the performance, we equip DAIL-SQL with self-consistency [44], which achieves a performance of 86.6% execution accuracy. Although self-consistency improves the execution accuracy by 0.4%, it is very time consuming and yields many times the cost of original DAIL-SQL. Therefore, in this paper we still focus on DAIL-SQL.

3.4 Supervised Fine-Tuning for Text-to-SQL

To enhance the performance of LLMs in zero-shot scenario, the popular option for existing Text-to-SQL methods is in-context learning, which is discuss in above subsections. As an alternative yet promising option, supervised fine-tuning is less explored so far. Similar to supervised fine-tuning for various language task, we can adopt it to the field of Text-to-SQL, and improve LLMs' performance on this downstream task. To further understand how supervised fine-tuning works for Text-to-SQL, we first provide a brief formulation as follows.

For Text-to-SQL, given a large language model $\mathcal{M}$, a set of Text-to-SQL training data $\mathcal{T} = \{(q_i, s_i, \mathcal{D}_i)\}$, where q_i and s_i are the natural language question and its corresponding query on database $\mathcal{D}_i$, the objective of supervised fine-tuning is to minimize the following empirical loss:

$$\min_{\sigma, \mathcal{M}^*} \sum_{i=1}^{|\mathcal{T}|} \mathcal{L}_{\mathcal{M}^*}(\sigma(q_i, \mathcal{D}_i), s_i),$$

where $\mathcal{L}$ is the loss function to measure the difference between the generated query and the groundtruh query. Similar to question representation, σ decides question representation with useful information from the schema in database $\mathcal{D}$. In this definition, supervised fine-tuning for Text-to-SQL covers two sub-tasks, including fine-tuning the given LLM $\mathcal{M}$ using supervised data $\mathcal{T}$ in order to get the optimal LLM $\mathcal{M}^*$, and searching for the optimal question representation σ. Since question representations have been discussed in Sec. 3.1, this section will primarily focus on data preparation $\mathcal{T}$ and fine-tuning.

For general domain, each item in supervised data $\mathcal{T} = \{(p_i, r_i)\}$ contains an input prompt p_i and an expected respond r_i from LLM. To ensure consistency with the inference process, we employ a supervised fine-tuning and generate prompt-response pairs from a given Text-to-SQL dataset. Specifically, given a Text-to-SQL data set $\mathcal{T} = \{(q_i, s_i, \mathcal{D}_i)\}$, we fine-tune the LLMs using the generated tuning data by using target question and the given database as prompt, and treating the desired query as response from LLM, i.e., $\mathcal{T} = \{(p_i = \sigma(q_i, \mathcal{D}_i), r_i = s_i)\}$. Once the data is ready, we can use existing package to fine-tune the given LLM $\mathcal{M}$ through either full fine-tuning [29] or parameter-efficient fine-tuning [13] depending on the available computational resources. After fine-tuning, the optimized LLM $\mathcal{M}^*$ can be used to do inference, that is asking it to generate queries through natural language questions. Note that we utilize the same question representation σ in both fine-tuning and inference processes. We will conduct a series of experiments and discuss the great potential of supervised fine-tuning for Text-to-SQL.

4 EXPERIMENT

In this section, we first introduce our experimental settings. Then we conduct extensive comparisons with existing solutions in question representation, in-context learning and supervised fine-tuning respectively. After that, we further compare them in terms of token efficiency to inspire more efficient solutions.

4.1 Setting

Dataset. We evaluate Text-to-SQL methods on two well recognized datasets, **Spider** [47] and **Spider-Realistic** [8]. Spider is a large-scale cross-domain Text-to-SQL dataset, which contains 8659 instances in training split and 1034 instances in development split over 200 databases. Each instance is consisted of a natural language question on a specific database and its corresponding SQL query. In this paper, we use the development split *Spider-dev* for the purpose of evaluation as the test split is not released. Spider-Realistic [8] is a more challenging variant of Spider. It selects a subset of 508 examples from Spider-dev and manually revises the questions while keeping the SQL queries unchanged. For few-shot scenarios, we utilize the training split of Spider as the example candidates when testing with both Spider-dev and Spider-Realistic.

Metric. To make a fair comparison, we follow prior study [51] to use exact-set-match accuracy (**EM**) and execution accuracy (**EX**). The exact-set-match accuracy measures the matched SQL keywords between the predicted SQL query and its corresponding ground truth. The execution accuracy, on the other hand, compares the execution output of the predicted SQL query with that of the ground truth SQL query on some database instances. This metric provides a more precise estimate of the model's performance since there may be multiple valid SQL queries for a single given question. We use the existing released evaluation scripts at https://github.com/taoyds/test-suite-sql-eval.

LLM. To ensure a fair comparison, for all the methods, we use the same maximal context length, that is 4096 for OpenAI LLM and 2048 for open-source LLM. During evaluation, we leave 200 tokens for response generation. By default, we set the argument temperature as 0 to eliminate the influence of randomness. Regarding post-processing, we follow existing work to extract the first SQL query in response and remove additional output. For more implementation details, please refer to Appendix A.1.

4.2 Question Representations

In this subsection, we evaluate the question representations presented in Sec. 3.1 under zero-shot scenario, employing three OpenAI LLMs: GPT-4, GPT-3.5-TURBO, and TEXT-DAVINCI-003.

Fig. 1 presents the comparison of different question representations over Spider-dev. By comparing different representations, we can observe that OD $_P$ fits to all three LLMs and achieves 75.5% execution accuracy with GPT-3.5-TURBO. In contrast, AS $_P$ exhibits poor performance with GPT-3.5-TURBO and TEXT-DAVINCI-003, necessitating a suitable LLM to work well with. Unexpectedly, GPT-4 exhibits a preference for the simple BS $_P$ derived from Din-SQL [31], indicating that a powerful LLM can mitigate the

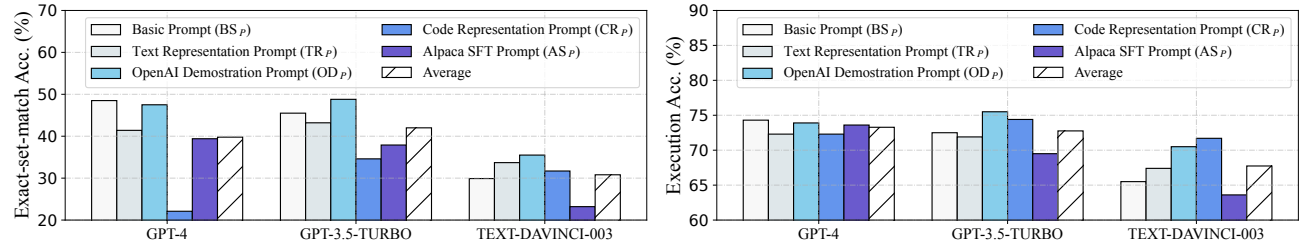


Figure 1: Results of different question representations on Spider-dev under zero-shot scenario.

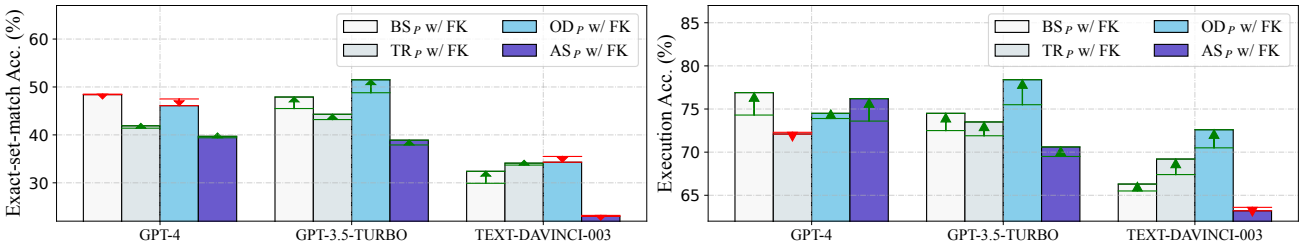


Figure 2: Ablation studies of foreign keys information on Spider-dev. The green arrow indicates an increase, and red arrow indicates a decrease.

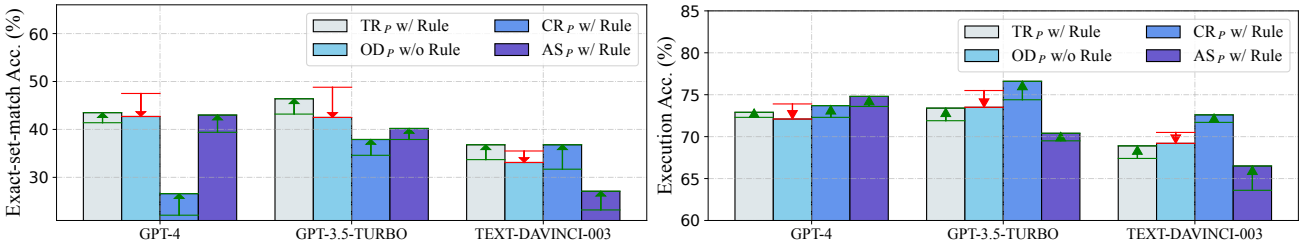


Figure 3: Ablation studies of "with no explanation" rule implication on Spider-dev. The green arrow indicates an increase, and red arrow indicates a decrease.

complexities associated with representation design. Besides, by comparing the average performance for three LLMs, GPT-4 and GPT-3.5-TURBO are more capable in the zero-shot scenario. Due to the expensive cost of GPT-4, GPT-3.5-TURBO together with OD $_P$ maybe a better choice for the zero-shot scenario. For detailed numerical results, please refer to Appendix A.2.

To further investigate the different question representations, we conduct ablation study to explore the effects of their invidual components.

Foreign Key (FK). Foreign Key implies the relation among different relational tables, which might be helpful in Text-to-SQL task. In our evaluation, only CR $_P$ contains foreign key information. To examine its effect, we add foreign key information into other representations and evaluate them in Fig. 2. We observe that foreign key information significantly improves the execution accuracy of LLMs by 0.6% to 2.9%, except the combinations of TR $_P$ with GPT-4 (−0.2%) and AS $_P$ with TEXT-DAVINCI-003 (−0.4%). Among all question representations, OD $_P$ benefits the most from foreign key information. Such observation demonstrates that foreign keys are helpful for Text-to-SQL task. For detailed performance, please refer to Appendix A.3.

Rule Implication (RI). Inspired by the outperformance of OD $_P$, we explore the effect of rule implication. Specifically, OD $_P$ implicate LLMs to generate SQL queries "*with no explanation*" To examine the effect of "*with no explanation*" rule in question representation, we present an ablation study in Fig. 3. Specifically, we remove "*with no explanation*" from OD $_P$, and add it to other representations. From Fig. 3 we observe adding this rule consistently booms the performance of all LLMs in both exact-set-match and execution accuracy, with the most significant improvements exceeding 6% and 2%, respectively. While for OD $_P$, removing this rule incurs about 2.4% − 6.2% drop in exact-set-match accuracy, and 1.0% − 1.8% drop in execution accuracy, indicating the importance of this rule implication. As a comparison, we also test a popular rule implication "*Let's think step by step*" [17], which guides LLM to generate response with analysis. However, its performance is highly volatile in Text-to-SQL task as Appendix A.5 shows. Due to limited resources, we leave the exploration of other possible rule implications as an open question for future research.

Few-shot	Selection	Question Similarity	Query Similarity	GPT-4		GPT-3.5-TURBO		TEXT-DAVINCI-003	
				EM	EX	EM	EX	EM	EX
0-shot	-	-	-	22.1	72.3	34.6	74.4	31.7	71.7
1-shot	Random	0.23	0.47	41.7	77.4	45.9	73.9	38.2	70.6
	Question Similarity Selection	0.39	0.65	53.3	78.8	51.9	74.3	44.1	72.3
	Masked Question Similarity Selection	0.57	0.80	58.2	79.1	57.4	76.0	47.9	75.0
	DAIL Selection	0.56	0.95	62.1	80.2	59.5	75.5	51.9	76.9
	Upper Limit	0.56	0.98	63.7	81.0	61.4	77.2	53.1	77.5
3-shot	Random	0.23	0.48	48.9	79.4	49.0	73.6	41.7	71.6
	Question Similarity Selection	0.37	0.63	56.3	79.2	53.8	74.7	52.2	74.1
	Masked Question Similarity Selection	0.54	0.78	66.1	81.5	61.1	77.3	59.7	77.0
	DAIL Selection	0.53	0.94	69.1	81.7	63.9	77.8	64.4	79.5
	Upper Limit	0.53	0.98	71.5	83.4	66.2	79.2	66.7	81.1
5-shot	Random	0.23	0.48	51.6	79.5	52.9	75.7	49.0	72.1
	Question Similarity Selection	0.36	0.61	58.2	79.9	55.9	75.1	54.8	73.2
	Masked Question Similarity Selection	0.52	0.77	66.8	82.0	62.3	77.9	64.7	78.6
	DAIL Selection	0.52	0.94	71.9	82.4	66.7	78.1	67.7	80.5
	Upper Limit	0.51	0.97	74.4	84.4	68.8	79.6	70.7	82.4

Table 2: Results of different selection strategies on Spider-dev with few-shot evaluation.

In summary, both the foreign key and the "*with no explanation*" implication rule are beneficial for Text-to-SQL task. In our evaluation, OD $_P$ with foreign keys and GPT-3.5-TURBO are the most effective and economic combination, which achieves 51.5% exact-set-match accuracy and 78.4% execution accuracy.

4.3 In-Context Learning for Text-to-SQL

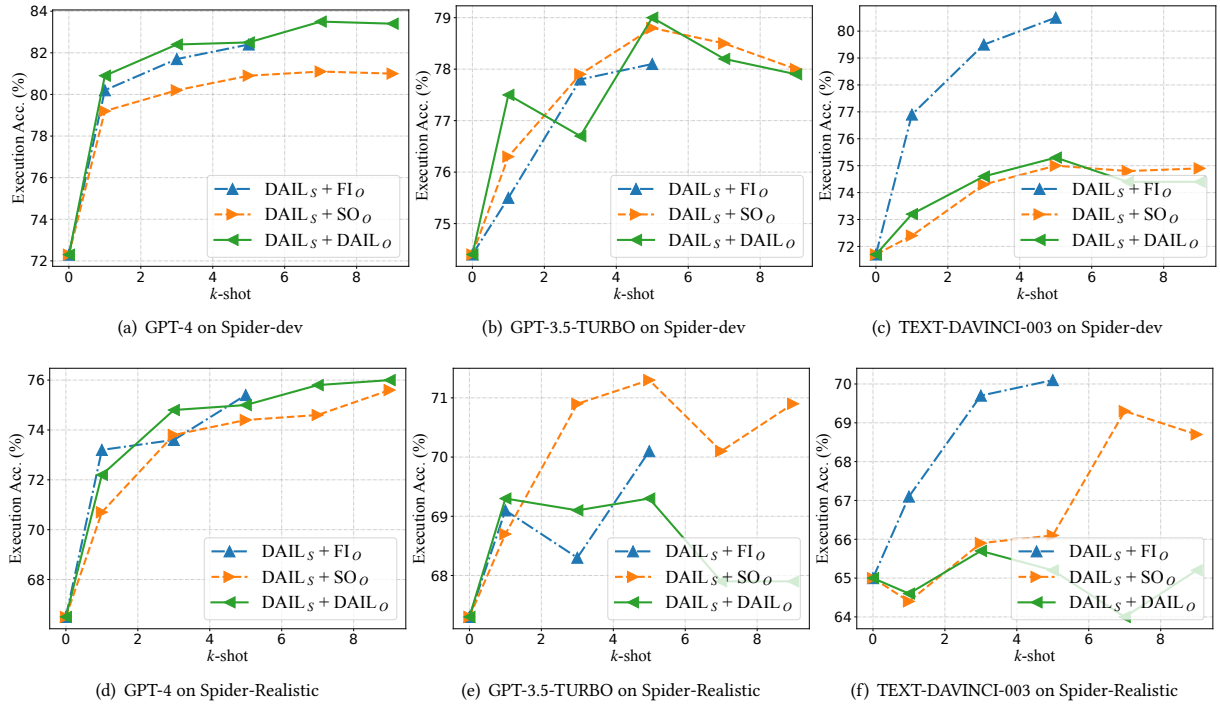
In few-shot scenario, we examine different example selection and organization strategies with GPT-4, GPT-3.5-TURBO, and TEXT-DAVINCI-003. To ensure a fair comparison, we adopt CR $_P$ as the question representation for all the experiments in this subsection, due to its superior performance in one-shot preliminary experiment in Appendix B.1.

4.3.1 Example Selection. To verify the importance of both question and query for example selection, we calculate question's and query's Jaccard similarities between chosen examples and the target instance, and report the averaged numbers under column *question similarity* and *query similarity* in Table 2. Specifically, we remove database-specific information from questions [42] and queries [19], and calculate the Jaccard similaritis of the remained tokens. Besides, we introduce **Upper Limit** for reference, which utilizes the ground truth query s^* rather than the query generated by preliminary predictor. To some extent, Upper Limit indicates the upper bound of performance for similarity based selection methods.

Table 2 shows the comparisons of different example selection strategies in 1-, 3- and 5-shot scenarios on Spider-dev. By comparing different selection strategies, it is demonstrated that DAIL $_S$ generally outperforms other strategies. In 5-shot scenario, equipped with GPT-4, DAIL-SQL achieves 82.4% execution accuracy. Besides, in Table 2 we observe the increasing question and query similarity corresponds to higher execution accuracy mostly, indicating the importance of considering both question and query similarity. Note DAIL $_S$'s execution accuracy is still lower than Upper Limit. This discrepancy can be attributed to the lower query similarity, indicating the gap between the ground truth query and that generated by the preliminary model.

4.3.2 Example Organization. To compare different example organization strategies, we evaluate Full-Information Organization, SQL-Only Organization and DAIL Organization in few-shot scenario on both Spider-dev and Spider-Realistic. Fig. 4 shows the comparison results, and refer to Appendix B.2 for detailed numerical results.

From Fig. 4(a) and Fig. 4(d), we can observe that GPT-4 benefits from contextual examples steadily on both Spider-dev and Spider-Realistic. With DAIL Organization, its execution accuracy increases from 72.3% to 83.5% on Spider-dev and from 66.5% to 76.0% on Spider-Realistic. While for GPT-3.5-TURBO and TEXT-DAVINCI-003, adding examples may incur drop in execution accuracy due to limited in-context learning capability. By comparing different organization strategies, we observe that GPT-4 shows preference for DAIL Organization in both Spider-dev and Spider-Realistic, suggesting it can effectively learn the mapping from question-SQL pairs. For GPT-3.5-TURBO (Fig. 4(b) and Fig. 4(e)), compared with its zero-shot performance in Fig. 1, its enhancement in in-context learning is the smallest among three LLMs, due to its weakness in in-context learning. For TEXT-DAVINCI-003, Full-Information Organization is far beyond the other two strategies, especially with increasing example number, as depicted in Fig. 4(c) and Fig. 4(f). By comparing different LLMs, we infer that for LLM with greater in-context learning capability, like GPT-4, benefits from DAIL Organization the most, while the weaker LLMs require more information to learn from examples. However, we emphasize DAIL Organization can be a good choice to achieve higher performance, and the best execution accuracy in our evaluation is achieved by DAIL Organization with GPT-4.



(a) GPT-4 on Spider-dev (b) GPT-3.5-TURBO on Spider-dev (c) TEXT-DAVINCI-003 on Spider-dev

(d) GPT-4 on Spider-Realistic (e) GPT-3.5-TURBO on Spider-Realistic (f) TEXT-DAVINCI-003 on Spider-Realistic

Figure 4: Results of few-shot evaluation with different example organizations.

In summary, for example selection, our findings emphasize the importance of the mapping from question to SQL query. Considering both question and query similarities simultaneously, DAIL $_S$ outperforms other selection strategies in our evaluation. For example organization, we show the effectiveness of DAIL $_O$, and point out its demands for potent LLMs. Finally, in our evaluation, we observe that our approach, DAIL-SQL, equipped with GPT-4, achieves the highest performance with an execution accuracy of 83.5% on Spider-dev and 76.0% on Spider-Realistic.

4.4 Supervised Fine-Tuning for Text-to-SQL

In this section, we investigate supervised fine-tuning in Text-to-SQL. Due to the unaffordable cost of fine-tuning OpenAI LLMs, we focus on open-source LLMs. Given the fact that very few existing work adopt open-source LLMs and their performance remain unknown, we first undertake a thorough evaluation for open-source LLMs, employing various question representation, example selection and organization strategies. After that, we fine-tune open-source LLMs in Text-to-SQL and observe their enhancement in both zero-shot and few-shot scenarios.

4.4.1 Open-source LLM. To investigate the potential of open-source LLM, we choose LLaMA [39], and its aligned variants in varying scales. They are detailed as follows. Note the aligned variants means the LLM is aligned to be more helpful, harmless and honest [2], and the suffix "-7B" means the LLM has 7 billions parameters, the same meaning for "-13B" and "-33B".

- **LLaMA-7B/13B/33B** [39] is a collection of widely recognized open-source LLMs, which are pre-trained on massive corpus by Meta.
- **Alpaca-7B** [38] is an aligned version of LLaMA-7B, which is fine-tuned with 52k instruction-following data generated by TEXT-DAVINCI-003.
- **GPT4ALL-7B** [1] is another aligned version of LLaMA-7B with about 800k data designed for helpful, harmless and honest AI assistant.
- **LLaMA-2-CHAT-7B/13B** [40] are up-to-date version of LLaMA. They are both pre-trained and aligned, and outperform the previous version on most benchmarks.
- **Vicuna-7/13/33B** [6, 49] is a collection of open-source chatbot aligned from LLaMA with user-shared conversations. Vicuna-13B [6] is declared to perform similar to OpenAI ChatGPT and Google Bard, and outperforms LLaMA and Alpaca in most scenarios.

4.4.2 Zero-shot Scenario with Open-source LLM. Table 3 shows their zero-shot performances on Spider-dev with different question representations. Due to limited space, please refer to Appendix C.1 for the performance on Spider-Realistic. Next, we provide several analysis from aspects of question representations, model scale and alignment as follows.

Effect of Question Representation. We can observe that the best performances is achieved by CR $_P$ with 43.7% execution accuracy on Spider-dev. The possible reason is that full database knowledge in CR $_P$ (Code Representation Prompt) compensates the incapability of open-source LLMs.

	LLM	BS $_P$		TR $_P$		OD $_P$		CR $_P$		AS $_P$		Average	
		EM	EX	EM	EX	EM	EX	EM	EX	EM	EX	EM	EX
Pre-trained	LLaMA-7B	6.5	9.6	3.1	4.9	3.6	9.0	4.8	16.3	1.3	5.9	3.9	9.1
	LLaMA-13B	8.8	18.4	4.5	15.2	8.2	21.8	5.6	25.0	8.9	26.9	7.2	21.5
	LLaMA-33B	9.6	26.7	12.0	25.9	13.6	36.4	12.2	**42.8**	**13.8**	38.1	12.2	34.0
Aligned	Alpaca-7B	15.1	25.1	13.5	23.8	14.7	25.7	16.0	32.1	8.9	19.9	13.6	25.3
	GPT4ALL-7B	7.8	19.4	8.8	24.6	8.1	27.0	8.5	25.9	6.5	21.8	7.9	23.7
	LLaMA-2-CHAT-7B	14.3	23.4	7.2	15.5	6.3	12.3	12.2	25.5	5.0	20.5	9.0	19.4
	LLaMA-2-CHAT-13B	18.8	32.6	15.4	30.5	11.1	22.3	**20.7**	40.0	16.9	36.2	16.6	32.3
	Vicuna-7B	7.5	15.6	1.2	9.9	6.2	21.5	5.6	24.0	0.9	5.4	4.3	15.3
	Vicuna-13B	8.2	21.7	10.1	24.4	11.2	31.4	5.8	33.5	4.7	20.0	8.0	26.2
	Vicuna-33B	10.8	28.9	18.3	37.1	19.1	42.7	6.9	**43.7**	8.6	30.6	12.7	36.6

Table 3: Zero-shot evaluation results on Spider-dev with different open-source LLMs. The best performances of pre-trained and aligned LLM are in bold.

Effect of Model Scale. From the results we observe a positive correlation between model scale and performance on Text-to-SQL for both LLaMA and Vicuna. Specifically, the average execution match accuracy of LLaMA shows a notable progression from 9.1% to 34.0% on Spider-dev, and Vicuna shows a similar upward trend from 15.3% to 36.6%. In the more challenging dataset Spider-Realistic, the same pattern can be observed and execution accuracy of LLaMA and Vicuna rise from 7.56% to 25.4% and 12.3% to 30.0%.

Effect of Alignment. From the results we observe that LLM alignment can benefit Text-to-SQL. Specifically, with the same model scale, Vicuna outperforms LLaMA by about 5% in execution accuracy on both Spider-dev and Spider-Realistic. Besides, the aligned LLMs share similar preference in question representation with the pre-trained ones, and their highest execution accuracies are all achieved with CR $_P$.

4.4.3 Few-shot Scenario with Open-source LLM. For few-shot scenario, Fig. 5 shows the performance of LLaMA-33B and Vicuna-33B with CR $_P$. We use DAIL Selection to select example as it is reported as the best strategy in Sec. 4.3. For more details, refer to Appendix C.2. From this Figure, we can see that LLaMA-33B benefits more than Vicuna-33B, and achieves 36.4% exact-set-match accuracy with 5-shot Full-Information Organization examples. Regarding execution match accuracy, increasing number of examples benefits Text-to-SQL in most cases. Besides, among different organization strategies, Full-Information Organization outperforms other strategies in different k-shot scenarios, which achieves 51.5% execution accuracy with Vicuna-33B.

Notably, in both zero-shot and few-shot scenarios, the open-source LLMs are far behind OpenAI LLMs. We will try to further enhance their performance with supervised fine-tuning.

4.4.4 Supervised Fine-tuning with Open-source LLM. To further enhance Open-source LLMs' performances, we explore supervised fine-tuning for Text-to-SQL. Similar to in-context learning, it may prefer different representations. Thus, we first fine-tune open-source LLMs on zero-shot training samples with different representations. Following the setting of supervised fine-tuning [29, 38], we block the gradients from prompt and only update weights with those from response (SQL queries). We use the train split in Spider,

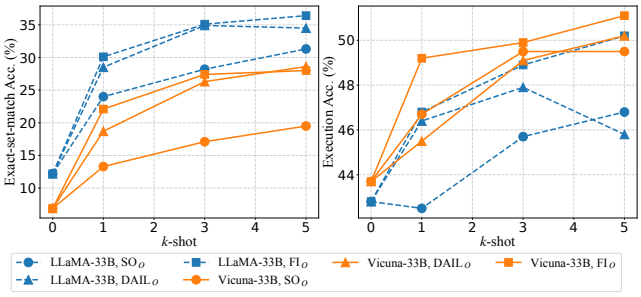


Figure 5: Few-shot evaluation with open-source LLMs on Spider-dev.

which contains 8659 training samples. For more training details, please refer to Appendix C.3.

Zero-shot Scenario. Fig. 6 shows the performance of supervised fine-tuning with various LLMs and question representations in zero-shot scenario. Compared with zero-shot performance before fine-tuning in Table 3 , their performances are greatly enhanced. By comparing different representations, Alpaca SFT Prompt show obvious advantages in supervised fine-tuning as it is designed for such scenario.

We also observe the gap among different representations and model scales becomes narrow. The possible reason is that after fine-tuning, LLM can generalize to evaluation samples even with weak representations, like Basic Prompt. In this experiment, the best performance on Spider is achieved by the combination of LLaMA-13B and Alpaca SFT Prompt, whose exact-set-match and execution accuracy are 65.1% and 68.6%. For more detailed numerical results, please refer to Appendix C.4. As for larger LLM, the combination of LLaMA-33B and Code Representation Prompt achieves 69.1% execution accuracy and 65.9% exact-set-match accuracy. Due to the limited resources, we leave LLMs larger than 33B as our future work.

In summary, supervised fine-tuning is quite beneficial for open-source LLMs in Text-to-SQL. Compared with OpenAI LLMs, in zero-shot scenario, fine-tuned LLaMA-13B and 30B are comparable

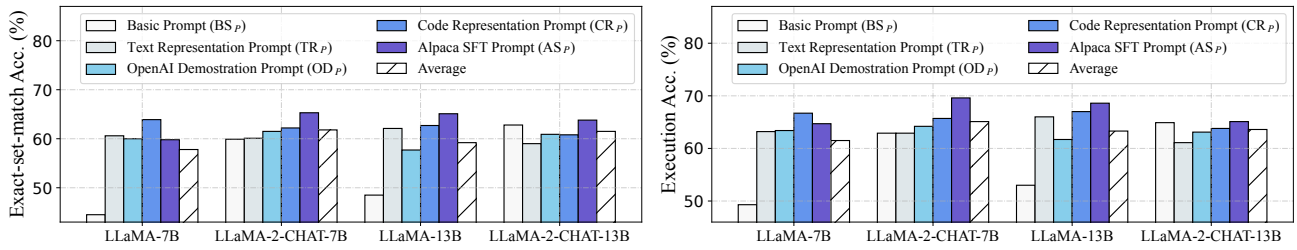


Figure 6: Zero-shot evaluation results on Spider-dev with different fine-tuned open-source LLMs.

LLM	Org.	0-shot		1-shot		3-shot		5-shot	
		EM	EX	EM	EX	EM	EX	EM	EX
LLaMA -7B	FI$_O$	3.1	13.0	23.4	30.1	23.7	30.3	24.7	30.9
	SO$_O$	3.1	13.0	13.3	21.4	15.2	24.1	15.3	25.0
	DAIL$_O$	3.1	13.0	18.5	25.4	22.1	28.1	22.6	29.3
+ SFT	FI$_O$	63.9	66.7	59.6	61.4	58.7	61.4	59.4	61.5
	SO$_O$	63.9	66.7	59.8	62.3	58.8	61.1	59.5	62.2
	DAIL$_O$	63.9	66.7	58.5	61.9	59.8	61.7	58.9	60.9
LLaMA -13B	FI$_O$	2.4	20.3	21.6	33.8	27.3	38.1	28.5	38.8
	SO$_O$	2.4	20.3	20.7	33.6	23.2	35.9	27.4	36.9
	DAIL$_O$	2.4	20.3	13.2	30.0	15.5	32.3	16.2	32.4
+ SFT	FI$_O$	62.7	67.0	61.9	67.1	60.5	65.0	60.9	65.0
	SO$_O$	62.7	67.0	61.9	66.2	60.1	64.6	60.2	65.2
	DAIL$_O$	62.7	67.0	62.5	66.5	60.6	66.0	61.3	66.4

Table 4: Few-shot evaluation results of supervised fine-tuned LLMs on Spider-dev.

to TEXT-DAVINCI-003 and slightly weaker than GPT-4 and GPT-3.5-TURBO.

Few-shot Scenario. After supervised fine-tuning, an important issue is: *Can we continue to enhance the performance of open-source LLM by adding contextual examples?* To answer this question, we evaluate fine-tuned LLaMA-7B and 13B with 0, 1, 3 and 5-shot prompts as shown in Table 4. We also add the evaluation results of original LLaMA-7B and 13B for clear comparison. Unexpectedly, the fine-tuned LLMs fail to learn from examples. Specifically, adding contextual examples in test prompts incurs sudden decrease in both exact-set-match and execution match accuracy, and adding more examples is also unhelpful. A possible reason is that LLM overfits to zero-shot prompt, which makes examples unuseful.

In summary, open-source LLMs demonstrate significant potential for Text-to-SQL tasks, particularly in supervised fine-tuning. Specifically, after fine-tuning, their performances are comparable to TEXT-DAVINCI-003 in zero-shot scenario. However, unlike OpenAI LLMs, fine-tuned LLMs fail to learn from contextual examples. The question of preserving in-context learning ability after fine-tuning remains to be explored in future studies.

4.5 Token Efficiency

Considering OpenAI LLMs are charged by token numbers, and LLMs' running time are proportional to token lengths, we underscore token efficiency in prompt engineering, which aims to achieve higher accuracy with less tokens. In this section, we review our experiments on Spider-dev in terms of token efficiency. Specifically, for both OpenAI and open-source LLMs, we experimentally study the trade-off between execution accuracy and token numbers, and the token number is mainly affected by question representation and example organization. For example selection, we fix it as DAIL$_S$. Besides, we also include several state-of-the-art Text-to-SQL methods in our comparison, including DIN-SQL [31], STRIKE [25] and CBR-ApSQL [11]. We take their reported highest execution accuracy as their performances. For token cost, we average the token number of 10 randomly sampled instances for DIN-SQL. For STRIKE, the optimal performance are achieved by majority voting from 1-shot to 5-shot results, resulting in a significant increase in token cost. Further, for CBR-ApSQL the token cost is calculated with their question representation and 8-shot examples in SQL-Only Organization.

Fig. 7 shows the comparison in terms of token efficiency. In zero-shot scenario, compared with rule implication, prompt with foreign keys generally achieve higher execution accuracy at the expense of more tokens. In few-shot scenario, comparing different organization strategies, FI$_O$ are very inefficient, whose tokens numbers are several times that of DAIL$_O$ and SO$_O$. Comparing DAIL$_O$ and SO$_O$, DAIL$_O$ together with GPT-4 achieve the highest accuracy of 83.5%, yet having similar token cost with SO$_O$. Therefore, we demonstrate DAIL$_O$ are more efficient than SO$_O$ and FI$_O$ in terms of token.

Compared with other state-of-the-art solutions, DAIL-SQL outperforms DIN-SQL and STRIKE in terms of both accuracy and efficiency. While for CBR-ApSQL, it achieves 78.2% accuracy with TEXT-DAVINCI-003, but still lower than the optimal performance achieved by DAIL$_S$ + FI$_O$.

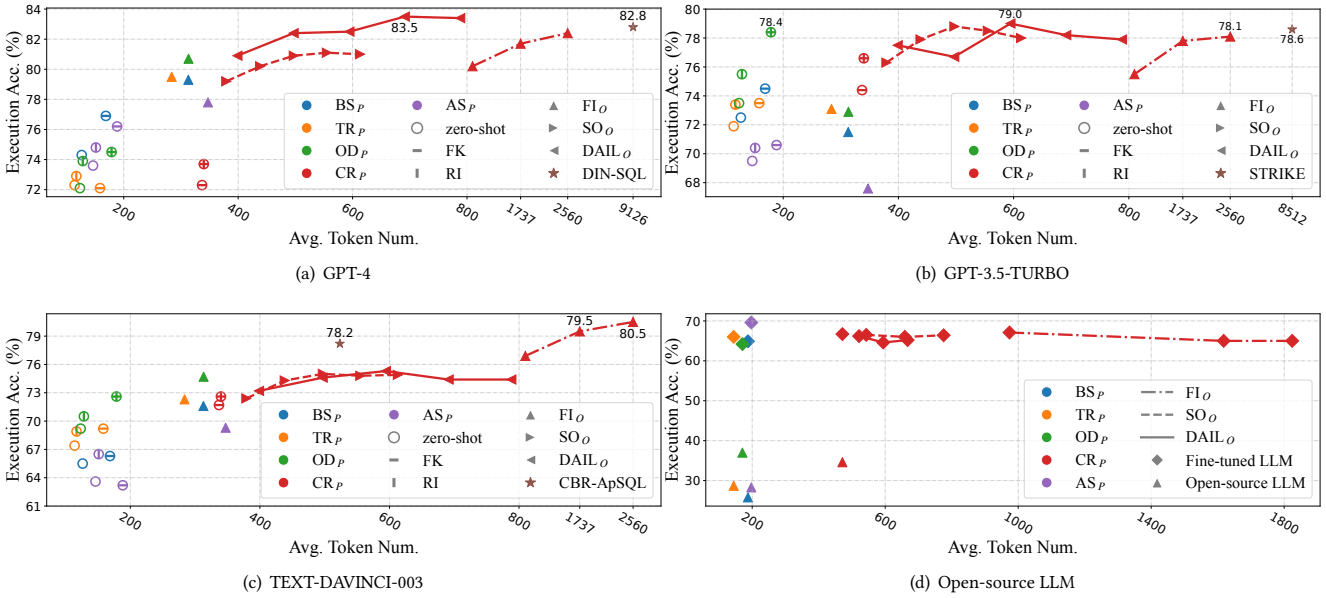
Besides, For open-source LLM in Fig. 7(d), the LLMs fine-tuned on Text-to-SQL are much more efficient. However, as discussed in Sec. 4.4, adding examples is unhelpful for open-source LLMs, and even reduces their token efficiency.

In summary, token efficiency is a critical metric for real-world applications of LLMs on Text-to-SQL. In light of this, our approach, DAIL-SQL, offers a compelling solution that combines high execution accuracy with improved token efficiency. This makes it highly practical and suitable for real-world applications.

5 DISCUSSION

Based on our experiments, we can have some empirical insights and guidelines as follows:

- For question representation, Code Representation Prompt and OpenAI Demostration Prompt are recommended, and



(a) GPT-4 (b) GPT-3.5-TURBO

(c) TEXT-DAVINCI-003 (d) Open-source LLM

Figure 7: Token efficiency of different representations in Spider-dev for OpenAI LLMs. We utilize different colors to represent different question representations and different shapes to denote different example organizations as well as the usage of foreign key information and rule implication. In particular, the overlap of shapes is used to indicate the usage of both foreign key information and rule implication. The rings stand for the prompts in zero-shot scenario and the stars stand for the previous SOTA results of few-shot methods in LLMs.

other information such as foreign key and rule implication can be very helpful.

- For example selection, the similarities of both natural language question and SQL query are important. These two similarities together are a good indicator for designing effective selection strategy.
- For example organization, if the adopted LLM is powerful enough, like GPT-4, presenting them question and SQL query pairs is an effective yet efficient choice. Otherwise, presenting them full information examples is suggested.
- For open-source LLM, supervised fine-tuning seems to be necessary, and Alpaca SFT Prompt used in Alpaca is suggested. Besides, to obtain comparable performance with OpenAI LLMs, larger LLMs might be helpful.

There are also some limitations in this paper. Due to limited resources, we only test two rule implications, and the exploration of more rules can further benefit LLM-based Text-to-SQL solutions. Besides, the databases in Spider and Spider-Realistic may be not large enough, and we believe some new challenges in effectiveness and efficiency will emerge if there are a mass of tables in Text-to-SQL task. Furthermore, the current evaluation metric focuses more on correctness rather than efficiency, and thus how to promote LLM to generate efficient SQL among all correct ones is an important yet unexplored question. We will keep working on these limitations and open questions.

6 CONCLUSIONS

In this paper, we conduct a systematical study on LLM-based Text-to-SQL from aspects of question representation, in-context learning and supervised fine-tuning. We point out that existing in-context learning techniques for Text-to-SQL neglect the mapping between questions and queries, as well as the trade-off between example quality and quantity. To address these issues, we proposed a new prompt engineering method, named DAIL-SQL, which refreshes the Spider leaderboard with 86.6% execution accuracy and ranks the first place. Regarding supervised fine-tuning, we demonstrate the great potentials of open-source LLMs for Text-to-SQL, underline the importance of question representation and model scaling, and point out the degeneracy of in-context learning capability after fine-tuning. Further, we conduct an observation over existing solutions in terms of token efficiency, which indicates DAIL-SQL is much more efficient and emphasizes the importance of token efficiency in prompt engineering. All of these are open challenges and opportunities for future study. We hope that our work can provide a comprehensive study about Text-to-SQL, give some guidelines for real-world applications, and help people advance its frontiers.

REFERENCES

[1] Yuvanesh Anand, Zach Nussbaum, Brandon Duderstadt, Benjamin Schmidt, and Andriy Mulyar. 2023. GPT4All: Training an Assistant-style Chatbot with Large Scale Data Distillation from GPT-3.5-Turbo. https://github.com/nomic-ai/gpt4all.

[2] Amanda Askell, Yuntao Bai, Anna Chen, Dawn Drain, Deep Ganguli, Tom Henighan, Andy Jones, Nicholas Joseph, Benjamin Mann, Nova DasSarma, Nelson Elhage, Zac Hatfield-Dodds, Danny Hernandez, Jackson Kernion, Kamal Ndousse, Catherine Olsson, Dario Amodei, Tom B. Brown, Jack Clark, Sam McCandlish, Chris Olah, and Jared Kaplan. 2021. A General Language Assistant as a Laboratory for Alignment. *CoRR* abs/2112.00861 (2021).

[3] LILY Group at Yale University. 2018. Spider 1.0, Yale Semantic Parsing and Text-to-SQL Challenge. https://yale-lily.github.io/spider.

[4] Ruichu Cai, Boyan Xu, Zhenjie Zhang, Xiaoyan Yang, Zijian Li, and Zhihao Liang. 2018. An Encoder-Decoder Framework Translating Natural Language to Database Queries. In *Proceedings of the Twenty-Seventh International Joint Conference on Artificial Intelligence*. 3977–3983.

[5] Shuaichen Chang and Eric Fosler-Lussier. 2023. How to Prompt LLMs for Text-to-SQL: A Study in Zero-shot, Single-domain, and Cross-domain Settings. *CoRR* abs/2305.11853 (2023).

[6] Wei-Lin Chiang, Zhuohan Li, Zi Lin, Ying Sheng, Zhanghao Wu, Hao Zhang, Lianmin Zheng, Siyuan Zhuang, Yonghao Zhuang, Joseph E. Gonzalez, Ion Stoica, and Eric P. Xing. 2023. Vicuna: An Open-Source Chatbot Impressing GPT-4 with 90%* ChatGPT Quality. https://lmsys.org/blog/2023-03-30-vicuna/

[7] Naihao Deng, Yulong Chen, and Yue Zhang. 2022. Recent Advances in Text-to-SQL: A Survey of What We Have and What We Expect. In *Proceedings of the 29th International Conference on Computational Linguistics*. 2166–2187.

[8] Xiang Deng, Ahmed Hassan Awadallah, Christopher Meek, Oleksandr Polozov, Huan Sun, and Matthew Richardson. 2021. Structure-Grounded Pretraining for Text-to-SQL. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. 1337–1350.

[9] Qingxiu Dong, Lei Li, Damai Dai, Ce Zheng, Zhiyong Wu, Baobao Chang, Xu Sun, Jingjing Xu, Lei Li, and Zhifang Sui. 2023. A Survey for In-context Learning. *CoRR* abs/2301.00234 (2023).

[10] Xuemei Dong, Chao Zhang, Yuhang Ge, Yuren Mao, Yunjun Gao, Lu Chen, Jinshu Lin, and Dongfang Lou. 2023. C3: Zero-shot Text-to-SQL with ChatGPT. *CoRR* abs/2307.07306 (2023).

[11] Chunxi Guo, Zhiliang Tian, Jintao Tang, Pancheng Wang, Zhihua Wen, Kang Yang, and Ting Wang. 2023. A Case-Based Reasoning Framework for Adaptive Prompting in Cross-Domain Text-to-SQL. *CoRR* abs/2304.13301 (2023).

[12] Jiaqi Guo, Zecheng Zhan, Yan Gao, Yan Xiao, Jian-Guang Lou, Ting Liu, and Dongmei Zhang. 2019. Towards Complex Text-to-SQL in Cross-Domain Database with Intermediate Representation. In *Proceedings of the 57th Conference of the Association for Computational Linguistics*. 4524–4535.

[13] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-Rank Adaptation of Large Language Models. In *The 10th International Conference on Learning Representations*.

[14] Binyuan Hui, Ruiying Geng, Lihan Wang, Bowen Qin, Yanyang Li, Bowen Li, Jian Sun, and Yongbin Li. 2022. S^2SQL: Injecting Syntax to Question-Schema Interaction Graph Encoder for Text-to-SQL Parsers. In *Findings of the Association for Computational Linguistics*. 1254–1262.

[15] George Katsogiannis-Meimarakis and Georgia Koutrika. 2023. A Survey on Deep Learning Approaches for Text-to-SQL. *VLDB J.* 32, 4 (2023), 905–936.

[16] Anirudh Khatry, Joyce Cahoon, Jordan Henkel, Shaleen Deep, K. Venkatesh Emani, Avrilia Floratou, Sumit Gulwani, Vu Le, Mohammad Raza, Sherry Shi, Mukul Singh, and Ashish Tiwari. 2023. From Words to Code: Harnessing Data for Program Synthesis from Natural Language. *CoRR* abs/2305.01598 (2023).

[17] Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. 2022. Large Language Models Are Zero-Shot Reasoners. *Advances in neural information processing systems* 35 (2022), 22199–22213.

[18] Chia-Hsuan Lee, Oleksandr Polozov, and Matthew Richardson. 2021. KaggleDBQA: Realistic Evaluation of Text-to-SQL Parsers. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing*. 2261–2273.

[19] Haoyang Li, Jing Zhang, Cuiping Li, and Hong Chen. 2023. RESDSQL: Decoupling Schema Linking and Skeleton Parsing for Text-to-SQL. In *37th AAAI Conference on Artificial Intelligence*. 13067–13075.

[20] Jinyang Li, Binyuan Hui, Reynold Cheng, Bowen Qin, Chenhao Ma, Nan Huo, Fei Huang, Wenyu Du, Luo Si, and Yongbin Li. 2023. Graphix-T5: Mixing Pre-trained Transformers with Graph-Aware Layers for Text-to-SQL Parsing. In *37th AAAI Conference on Artificial Intelligence*. 13076–13084.

[21] Jinyang Li, Binyuan Hui, Ge Qu, Binhua Li, Jiaxi Yang, Bowen Li, Bailin Wang, Bowen Qin, Rongyu Cao, Ruiying Geng, Nan Huo, Xuanhe Zhou, Chenhao Ma, Guoliang Li, Kevin Chen-Chuan Chang, Fei Huang, Reynold Cheng, and Yongbin Li. 2023. Can LLM Already Serve as A Database Interface? A BIg Bench for Large-Scale Database Grounded Text-to-SQLs. *CoRR* abs/2305.03111 (2023).

[22] Aiwei Liu, Xuming Hu, Lijie Wen, and Philip S. Yu. 2023. A Comprehensive Evaluation of ChatGPT's Zero-Shot Text-to-SQL Capability. *CoRR* abs/2303.13547 (2023).

[23] Hu Liu, Yuliang Shi, Jianlin Zhang, Xinjun Wang, Hui Li, and Fanyu Kong. 2023. Multi-hop Relational Graph Attention Network for Text-to-SQL Parsing. In *International Joint Conference on Neural Networks*. 1–8.

[24] Jiachang Liu, Dinghan Shen, Yizhe Zhang, Bill Dolan, Lawrence Carin, and Weizhu Chen. 2022. What Makes Good In-Context Examples for GPT-3?. In *Proceedings of Deep Learning Inside Out: The 3rd Workshop on Knowledge Extraction and Integration for Deep Learning Architectures*. 100–114.

[25] Linyong Nan, Yilun Zhao, Weijin Zou, Narutatsu Ri, Jaesung Tae, Ellen Zhang, Arman Cohan, and Dragomir Radev. 2023. Enhancing Few-shot Text-to-SQL Capabilities of Large Language Models: A Study on Prompt Design Strategies. *CoRR* abs/2305.12586 (2023).

[26] OpenAI. 2023. GPT-4 Technical Report. *CoRR* abs/2303.08774 (2023).

[27] OpenAI. 2023. Rate limits. https://platform.openai.com/docs/guides/rate-limits/overview. Last accessed on 2023-07-24.

[28] OpenAI. 2023. SQL translate. https://platform.openai.com/examples/default-sql-translate. Last accessed on 2023-07-24.

[29] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul F. Christiano, Jan Leike, and Ryan Lowe. 2022. Training Language Models to Follow Instructions with Human Feedback. In *NeurIPS*.

[30] Octavian Popescu, Irene Manotas, Ngoc Phuoc An Vo, Hangu Yeo, Elahe Khorashani, and Vadim Sheinin. 2022. Addressing Limitations of Encoder-Decoder Based Approach to Text-to-SQL. In *Proceedings of the 29th International Conference on Computational Linguistics*. 1593–1603.

[31] Mohammadreza Pourreza and Davood Rafiei. 2023. DIN-SQL: Decomposed In-Context Learning of Text-to-SQL with Self-Correction. *CoRR* abs/2304.11015 (2023).

[32] Jiexing Qi, Jingyao Tang, Ziwei He, Xiangpeng Wan, Yu Cheng, Chenghu Zhou, Xinbing Wang, Quanshi Zhang, and Zhouhan Lin. 2022. RASAT: Integrating Relational Structures into Pretrained Seq2Seq Model for Text-to-SQL. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*. 3215–3229.

[33] Bowen Qin, Binyuan Hui, Lihan Wang, Min Yang, Jinyang Li, Binhua Li, Ruiying Geng, Rongyu Cao, Jian Sun, Luo Si, Fei Huang, and Yongbin Li. 2022. A Survey on Text-to-SQL Parsing: Concepts, Methods, and Future Directions. *CoRR* abs/2208.13629 (2022).

[34] Nitarshan Rajkumar, Raymond Li, and Dzmitry Bahdanau. 2022. Evaluating the Text-to-SQL Capabilities of Large Language Models. *CoRR* abs/2204.00498 (2022).

[35] Torsten Scholak, Nathan Schucher, and Dzmitry Bahdanau. 2021. PICARD: Parsing Incrementally for Constrained Auto-Regressive Decoding from Language Models. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*. 9895–9901.

[36] Kaitao Song, Xu Tan, Tao Qin, Jianfeng Lu, and Tie-Yan Liu. 2020. MPNet: Masked and Permuted Pre-training for Language Understanding. In *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems*.

[37] Ruoxi Sun, Sercan Ö. Arik, Hootan Nakhost, Hanjun Dai, Rajarishi Sinha, Pengcheng Yin, and Tomas Pfister. 2023. SQL-PaLM: Improved Large Language Model Adaptation for Text-to-SQL. *CoRR* abs/2306.00739 (2023).

[38] Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B. Hashimoto. 2023. Stanford Alpaca: An Instruction-following LLaMA model. https://github.com/tatsu-lab/stanford_alpaca.

[39] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurélien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. 2023. LLaMA: Open and Efficient Foundation Language Models. *CoRR* abs/2302.13971 (2023).

[40] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, Dan Bikel, Lukas Blecher, Cristian Canton Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madian Khabsa, Isabel Kloumann, Artem Korenev, Singh Koura, Marie-Anne Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rashi Rungta, Kalyan Saladi, Alan Schelten, Ruan Silva, Eric Michael, Smith Ranjan, Subramanian Xiaoqing, Ellen Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xiang Kuan, Puxin Xu, Zheng Yan, Iliyan Zarov, Yuchen Zhang, Angela Fan, Melanie Kambadur, Sharan Narang, Aurelien Rodriguez, Robert Stojnic, Sergey Edunov, and Thomas Scialom. 2023. LLAMA2:

Open Foundation and Fine-Tuned Chat Models. *CoRR* (2023).

[41] Immanuel Trummer. 2022. CodexDB: Synthesizing Code for Query Processing from Natural Language Instructions Using GPT-3 Codex. *Proceedings of the VLDB Endowment* 15, 11 (2022), 2921–2928.

[42] Bailin Wang, Richard Shin, Xiaodong Liu, Oleksandr Polozov, and Matthew Richardson. 2020. RAT-SQL: Relation-Aware Schema Encoding and Linking for Text-to-SQL Parsers. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. 7567–7578.

[43] Lihan Wang, Bowen Qin, Binyuan Hui, Bowen Li, Min Yang, Bailin Wang, Binhua Li, Jian Sun, Fei Huang, Luo Si, and Yongbin Li. 2022. Proton: Probing Schema Linking Information from Pre-trained Language Models for Text-to-SQL Parsing. In *The 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 1889–1898.

[44] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V. Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2023. Self-Consistency Improves Chain of Thought Reasoning in Language Models. In *The Eleventh International Conference on Learning Representations*.

[45] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, and Jamie Brew. 2019. HuggingFace's Transformers: State-of-the-art Natural Language Processing. *CoRR* abs/1910.03771 (2019).

[46] Kun Xu, Lingfei Wu, Zhiguo Wang, Yansong Feng, and Vadim Sheinin. 2018. SQL-to-Text Generation with Graph-to-Sequence Model. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*. 931–936.

[47] Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, Zilin Zhang, and Dragomir R. Radev. 2018. Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*. 3911–3921.

[48] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, Yifan Du, Chen Yang, Yushuo Chen, Zhipeng Chen, Jinhao Jiang, Ruiyang Ren, Yifan Li, Xinyu Tang, Zikang Liu, Peiyu Liu, Jian-Yun Nie, and Ji-Rong Wen. 2023. A Survey of Large Language Models. *CoRR* abs/2303.18223 (2023).

[49] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. *CoRR* abs/2306.05685 (2023).

[50] Yanzhao Zheng, Haibin Wang, Baohua Dong, Xingjun Wang, and Changshan Li. 2022. HIE-SQL: History Information Enhanced Network for Context-Dependent Text-to-SQL Semantic Parsing. In *Findings of the Association for Computational Linguistics*. 2997–3007.

[51] Ruiqi Zhong, Tao Yu, and Dan Klein. 2020. Semantic Evaluation for Text-to-SQL with Distilled Test Suites. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*. 396–411.

[52] Victor Zhong, Caiming Xiong, and Richard Socher. 2017. Seq2SQL: Generating Structured Queries from Natural Language using Reinforcement Learning. *CoRR* abs/1709.00103 (2017).

A QUESTION REPRESENTATIONS

A.1 Implementation Details

For question similarity in QTS$_S$, MQS$_S$ and DAIL$_S$, we first connect question words to the database with a n-gram matching based schema-linking method [42]. Then to obtain the skeleton, we replace table and column names with "<mask>", and values with "<unk>". At last, we embed the masked questions with a pre-trained sentence Transformer, all-mpnet-base-v2 [36], to calculate their similarities.

For query similarity in DAIL$_S$, we utilize Graphix [20] as the preliminary model to generate the predicted query s'. Then we obtain its skeleton by removing its database-specific information [19], including column names and values. Finally, we calculate the Jaccard similarity between example candidate and the predicted query s' as their query similarity. For the query similarity threshold τ in DAIL$_S$, we set it as 0.9 in the experiments of this paper.

For the submission to the Spider leaderboard, we set τ to be 0.85 and utilize GPT-4, with CR$_P$, MQS$_S$ and DAIL$_O$, as the preliminary model to ensure only one model involved. Furthermore, we process the self-consistency voting on 5 produced queries for each question and set the argument temperature as 1.0 for variety in voting.

A.2 Detailed Performance of Different Question Representations

The numerical results of different question representations in zero-shot scenario are show in Table 5.

Prompt	GPT-4		GPT-3.5-TURBO		TEXT-DAVINCI-003	
	EM	EX	EM	EX	EM	EX
Basic Prompt	48.5	**74.3**	45.5	72.5	29.9	65.5
Text Representation Prompt	41.4	72.3	43.2	71.9	33.7	67.4
OpenAI Demostration Prompt	47.5	73.9	48.8	**75.5**	35.5	70.5
Code Representation Prompt	22.1	72.3	34.6	74.4	31.7	**71.7**
Alpaca SFT Prompt	39.4	73.6	37.9	69.5	23.2	63.6

Table 5: Details of zero-shot evaluation on Spider-dev with different question representations.

A.3 Detailed Performance of Different Representations with Foreign Keys

The numerical results of the ablation study about foreign key information are shown in Table 6.

Prompt	GPT-4		GPT-3.5-TURBO		TEXT-DAVINCI-003	
	EM	EX	EM	EX	EM	EX
Basic Prompt With Foreign Keys	48.4(-0.1)	76.9(+2.6)	47.9(+2.4)	74.5(+2.0)	32.4(+2.5)	66.3(+0.8)
Text Representation Prompt With Foreign Keys	41.9(+0.5)	72.1(-0.2)	44.3(+1.1)	73.5(+1.6)	34.1(+0.4)	69.2(+1.8)
OpenAI Demostration Prompt With Foreign Keys	46.1(-1.4)	74.5(+0.6)	51.5(+2.7)	78.4(+2.9)	34.3(-1.2)	72.6(+2.1)
Alpaca SFT Prompt With Foreign Keys	39.7(+0.3)	76.2(+2.6)	38.9(+1.0)	70.6(+1.1)	23.0(-0.2)	63.2(-0.4)

Table 6: Details of zero-shot evaluation on Spider-dev with foreign keys and comparisons with the results obtained without foreign keys in Table 5.

Prompt	GPT-4		GPT-3.5-TURBO		TEXT-DAVINCI-003	
	EM	EX	EM	EX	EM	EX
Text Representation Prompt With Rule	43.5(+2.1)	72.9(+0.6)	46.4(+3.2)	73.4(+1.5)	36.8(+3.1)	68.9(+1.5)
OpenAI Demostration Prompt Without Rule	42.7(-4.8)	72.1(-1.8)	**42.5(-6.3)**	73.5(-2.0)	33.1(-2.4)	69.2(-1.3)
Code Representation Prompt With Rule	26.6(+4.5)	73.7(+1.4)	37.9(+3.3)	76.6(+2.2)	36.8(+5.1)	72.6(+0.9)
Alpaca SFT Prompt With Rule	43.0(+3.6)	74.8(+1.2)	40.2(+2.3)	70.4(+0.9)	27.1(+3.9)	**66.5(+2.9)**

Table 7: Details of zero-shot evaluation on Spider-dev with/without rule implication "with no explanation" in instructions and comparisons with their opposites in Table 5.

A.4 Detailed Performance of Different Representations with/without Explanation rule

The numerical results of ablation study about rule implication are shown in Table 7.

A.5 Question Representations with Rule Implication "Let's think step by step"

The numerical results of the ablation study with the rule "Let's think step by step" are shown in Table 8.

Prompt	GPT-3.5-TURBO		TEXT-DAVINCI-003	
	EM	EX	EM	EX
Text Representation Prompt With Rule	20.0(-23.2)	45.9(-26.0)	23.0(-10.7)	46.1(-21.3)
Code Representation Prompt With Rule	21.7(-12.9)	52.2(-22.2)	31.9(+0.2)	63.8(-7.9)
OpenAI Demostration Prompt With Rule	48.4(-0.4)	75.8(+0.3)	41.2(+5.7)	72.4(+1.9)
Alpaca SFT Prompt With Rule	21.5(-16.4)	49.3(-20.2)	27.9(+4.7)	64.4(+0.8)

Table 8: Zero-shot evaluation results on Spider-dev with "Let's think step by step" rule implication in instructions and comparisons with Table 5.

B IN-CONTEXT LEARNING FOR TEXT-TO-SQL

B.1 One-Shot Evaluation on Different Question Representation

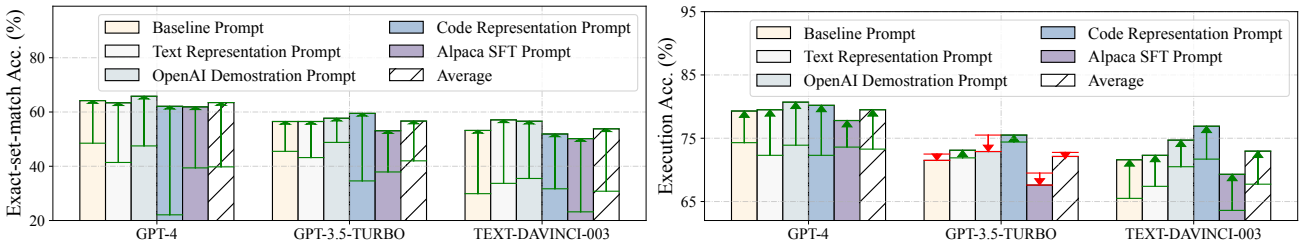


Figure 8: Results of one-shot evaluation on Spider-dev with different question representations and comparisons with the results of zero-shot evaluation in Fig. 1. The green arrow indicates increase, and red arrow indicates decrease.

Prompt	GPT-4		GPT-3.5-TURBO		TEXT-DAVINCI-003	
	EM	EX	EM	EX	EM	EX
Basic Prompt	64.2(+15.7)	79.3(+5.0)	56.5(+11.0)	71.5(-1.0)	53.2(+23.3)	71.6(+6.1)
Text Representation Prompt	63.4(+22.0)	79.5(+7.2)	56.5(+13.3)	73.1(+1.2)	57.1(+23.4)	72.3(+4.9)
OpenAI Demostration Prompt	65.8(+18.3)	**80.7(+6.8)**	57.7(+8.9)	72.9(-2.6)	56.6(+21.1)	74.7(+4.2)
Code Representation Prompt	62.1(+40.0)	80.2(+7.9)	59.5(+24.9)	**75.5(+1.1)**	51.9(+20.2)	**76.9(+5.2)**
Alpaca SFT Prompt	61.9(+22.5)	77.8(+4.2)	53.1(+15.2)	67.6(-1.9)	50.2(+27.0)	69.3(+5.7)

Table 9: Details of one-shot evaluation on Spider-dev with different question representations and comparisons with the results of zero-shot evaluation in Table 5.

In Fig. 8, we present the results of our one-shot evaluation for different question representations. Specifically, we use FI_O examples with DAIL_S here, and Table 9 shows the comparisons with the zero-shot scenario. By comparing zero-shot and one-shot evaluation results, adding contextual example show obvious and consistent improvements for all LLMs in exact-set-match accuracy. In term of execution accuracy, contextual examples benefits both GPT-4 and TEXT-DAVINCI-003. However, for GPT-3.5-TURBO, adding contextual examples only benefits TR_P and CR_P, indicating the in-context learning capability bias in different LLMs. By comparing different representations, CR_P shows obvious advantage in execution accuracy, as taking the advantage of programming.

B.2 Detailed Performance of Different Example Organizations

The numerical results of different example organization strategies in few-shot scenario are shown in Table 10 and 11.

Few-shot	Presentation	GPT-4		GPT-3.5-TURBO		TEXT-DAVINCI-003	
		EM	EX	EM	EX	EM	EX
0-shot	-	22.1	72.3	34.6	74.4	31.7	71.7
1-shot	Full-Information Organization	62.1	80.2	59.5	75.5	51.9	76.9
	SQL-Only Organization	55.2	79.2	51.2	76.3	41.2	72.4
	DAIL Organization	62.9	80.9	57.6	77.5	46.9	73.2
3-shot	Full-Information Organization	69.1	81.7	63.9	77.8	64.4	79.5
	SQL-Only Organization	64.7	80.2	56.2	77.9	48.6	74.3
	DAIL Organization	69.0	82.4	61.9	76.7	54.0	74.6
5-shot	Full-Information Organization	71.9	82.4	66.7	78.1	67.7	80.5
	SQL-Only Organization	66.6	80.9	56.2	78.8	52.1	75.0
	DAIL Organization	70.8	82.5	64.3	79.0	58.2	75.3
7-shot	SQL-Only Organization	67.8	81.1	57.2	78.5	52.1	74.8
	DAIL Organization	72.5	83.5	65.6	78.2	59.0	74.4
9-shot	SQL-Only Organization	67.6	81.0	57.7	78.0	53.0	74.9
	DAIL Organization	72.8	83.4	65.3	77.9	60.3	74.4

Table 10: Details of Few-shot evaluation on Spider development split with different organizations.

Few-shot	Presentation	GPT-4		GPT-3.5-TURBO		TEXT-DAVINCI-003	
		EM	EX	EM	EX	EM	EX
0-shot	-	19.9	66.5	29.3	67.3	28.0	65.0
1-shot	Full-Information Organization	50.2	73.2	48.6	69.1	42.1	67.1
	SQL-Only Organization	45.3	70.7	42.7	68.7	34.6	64.4
	DAIL Organization	51.2	72.2	48.4	69.3	38.6	64.6
3-shot	Full-Information Organization	59.1	73.6	52.6	68.3	54.5	69.7
	SQL-Only Organization	56.9	73.8	48.8	70.9	42.7	65.9
	DAIL Organization	60.8	74.8	52.8	69.1	48.4	65.7
5-shot	Full-Information Organization	63.8	75.4	58.7	70.1	57.7	70.1
	SQL-Only Organization	58.9	74.4	49.2	71.3	45.3	66.1
	DAIL Organization	60.0	75.0	55.3	69.3	51.4	65.2
7-shot	SQL-Only Organization	59.6	74.6	50.0	70.1	49.0	69.3
	DAIL Organization	63.6	75.8	57.1	67.9	51.6	64.0
9-shot	SQL-Only Organization	60.0	75.6	49.8	70.9	48.8	68.7
	DAIL Organization	63.8	76.0	57.5	67.9	53.7	65.2

Table 11: Details of Few-shot evaluation on Spider-Realistic dataset with different organizations.

C SUPERVISED FINE-TUNING FOR TEXT-TO-SQL

C.1 Detailed Performance of Open-source LLMs on Spider-Realistic

The numerical results are shown in Table 12.

C.2 Detailed Performance of Open-source LLMs on Spider-dev in Few-shot Scenario

The numerical results are shown in Table 13.

C.3 Details for Supervised Fine-tuning

For dataset, we use the train split in Spider, which totally contains 8659 training samples. For hyper-parameters, we set global batch size as 256, and search learning rate in $[1e-6, 1e-4]$ and weight decay in $\{1, 0.1, 0.01, 0\}$. During fine-tuning, we use a cosine learning rate scheduler in transformers [45] with a warm ratio 0.03. Besides, all LLMs are fine-tuned on a server with eight 64G A100 GPUs.

C.4 Detailed Performance of Fine-tuned Open-source LLM on Spider and Spider-Realistic

The numerical results on Spider-dev and Spider-Realistic are all shown in Table 14.

Stage	LLM	BS $_P$		TR $_P$		OD $_P$		CR $_P$		AS $_P$		Average	
		EM	EX	EM	EX	EM	EX	EM	EX	EM	EX	EM	EX
Pre-training	LLaMA-7B	4.7	12.0	1.4	4.9	1.4	5.1	3.1	13.0	0.2	2.8	2.2	7.6
	LLaMA-13B	5.9	15.7	3.3	13.6	4.3	17.9	2.4	20.3	3.1	13.8	3.8	16.3
	LLaMA-33B	7.1	17.5	8.3	21.5	9.6	28.3	7.9	**34.6**	**10.8**	25.2	8.7	25.4
Aligned	Alpaca-7B	7.7	22.8	9.6	19.3	11.4	21.7	12.8	26.0	0.8	6.9	8.5	19.3
	GPT4ALL-7B	3.5	12.6	7.7	19.3	6.1	18.5	7.5	17.1	1.4	6.5	5.2	14.8
	LLaMA-2-CHAT-7B	11.4	21.7	5.1	12.0	7.5	14.4	7.5	17.7	3.7	13.6	7.0	15.9
	LLaMA-2-CHAT-13B	14.4	25.8	12.8	26.4	11.6	22.0	**17.9**	32.9	15.9	28.3	14.5	27.1
	Vicuna-7B	6.7	18.3	0.8	8.1	4.7	16.3	3.9	14.0	0.6	4.9	3.3	12.3
	Vicuna-13B	6.9	19.3	4.9	16.1	8.5	25.2	4.3	27.6	4.1	15.0	5.7	20.6
	Vicuna-33B	8.1	20.7	13.8	28.7	16.9	**37.0**	5.1	34.3	8.7	27.6	10.5	30.0

Table 12: Zero-shot evaluation results on Spider-Realistic with different open-source LLMs.

Example Organization	LLM	0-shot		1-shot		3-shot		5-shot	
		EM	EX	EM	EX	EM	EX	EM	EX
SQL-Only Organization	LLaMA-33B	12.2	42.8	24.0	42.5	28.2	45.7	31.3	46.8
	Vicuna-33B	6.9	43.7	13.3	46.7	17.1	49.5	19.5	49.5
DAIL Organization	LLaMA-33B	12.2	42.8	28.5	46.4	34.9	47.9	34.5	45.8
	Vicuna-33B	6.9	43.7	18.7	45.5	26.3	49.1	28.6	50.2
Full-Information Organization	LLaMA-33B	12.2	42.8	30.1	46.8	35.1	48.9	36.4	50.2
	Vicuna-33B	6.9	43.7	22.1	49.2	27.4	49.9	28.0	51.1

Table 13: Detailed performance of open-source LLMs on Spider-dev in few-shot scenario.

LLM	Metric	Spider						Spider-Realistic					
		BS $_P$	TR $_P$	OD $_P$	CR $_P$	AS $_P$	Average	BS $_P$	TR $_P$	OD $_P$	CR $_P$	AS $_P$	Average
LLaMA-7B	EM	44.5	60.6	60.0	**63.9**	59.8	57.8	33.3	45.5	43.9	**51.6**	46.5	44.2
	EX	49.3	63.2	63.4	**66.7**	64.7	61.5	35.0	48.8	47.4	**53.7**	46.5	46.3
LLaMA-2-CHAT-7B	EM	59.9	60.1	61.5	62.2	**65.3**	61.8	48.2	45.7	45.3	49.4	**52.2**	48.2
	EX	62.9	62.9	64.2	65.7	**69.6**	65.1	51.4	44.5	47.4	49.6	**54.9**	49.6
LLaMA-13B	EM	48.5	62.1	57.7	62.7	**65.1**	59.2	39.0	50.4	48.2	48.2	**52.2**	47.6
	EX	53.0	66.0	61.7	67.0	**68.6**	63.3	44.7	54.1	51.6	52.6	**54.7**	51.5
LLaMA-2-CHAT-13B	EM	62.8	59.0	60.9	60.8	**63.8**	61.5	**53.7**	47.0	47.2	49.6	**53.7**	50.2
	EX	64.9	61.1	63.1	63.8	**65.1**	63.6	**54.5**	47.6	48.6	51.2	52.8	50.9

Table 14: Performance of supervised fine-tuning on Spider and Spider-Realistic with respect to different representations and LLMs.

D SPIDER LEADERBOARD

Fig. 9 shows the performance rank in Spider leaderboard on Sep 19, 2023. In the leaderboard, our solution DAIL-SQL with GPT-4 is reported to achieve 86.2% execution accuracy; further, with self-consistency, our solution achieves 86.6% execution accuracy, ranked as the first place.

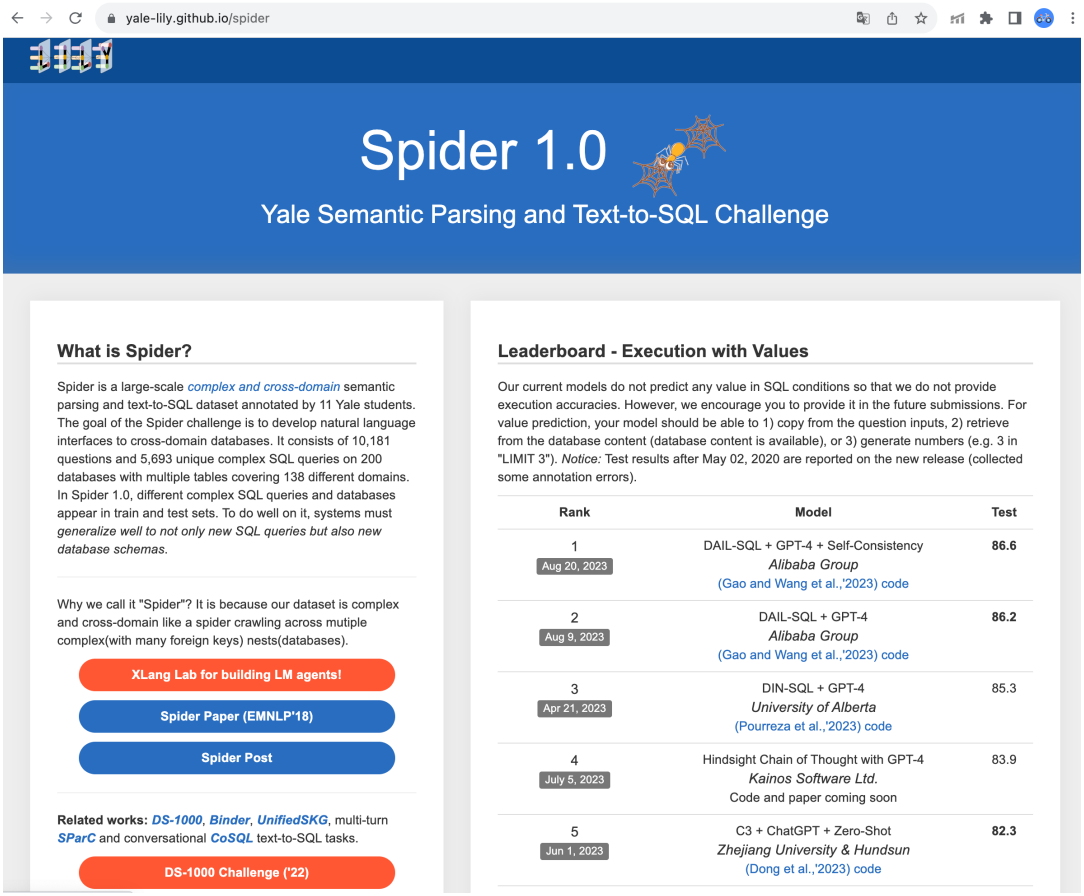


Figure 9: Current performance rank in Spider leaderboard. (Last accessed on 2023-09-19.)